

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

Transformers

EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Review 2
Going Deeper 19
Transformers 36
Positional Encoding 40
Applications 61
Generative Pre-Training 73
Closing Remarks 92

Review

Review

Last time, we derived various forms of **attention**

We started with composite memory

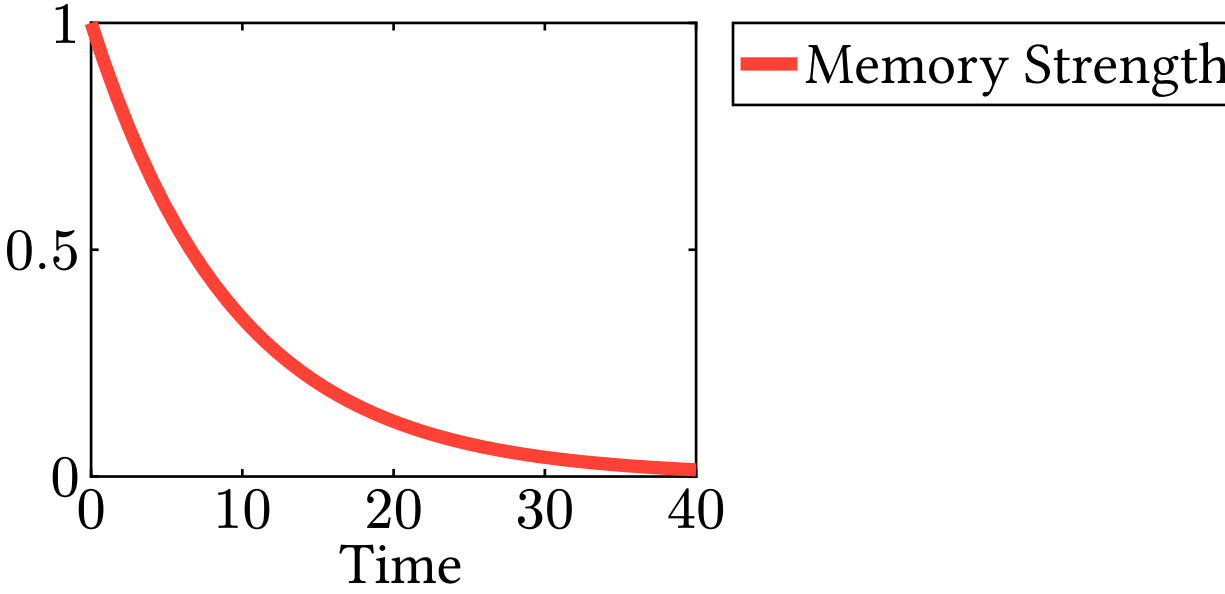
$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{i=1}^{T} \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_i$$

Given large enough T, we will eventually run out of storage space

Review

So we introduced a forgetting term γ

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sum_{i=1}^{T} \gamma^{T-i} \cdot \boldsymbol{\theta}^{\top} \overline{\boldsymbol{x}}_i$$



Review

We went to a party and the forgetting seemed ok



10 PM 11 PM 12 AM 1 AM

Review



$\gamma^3 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1$ $\gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2$ $\gamma^1 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3$ $\gamma^0 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_4$

Review

But we encountered problems when Taylor Swift arrived at the party



$$\gamma^4 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1 \quad \gamma^3 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2 \quad \gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3 \quad \gamma^1 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_4 \quad \gamma^0 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_5$$

Review



$\gamma^4 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1$ $\gamma^3 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2$ $\gamma^2 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3$ $\gamma^1 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_4$ $\gamma^0 \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_5$

With composite memory, we forget Taylor Swift!

Our model of human memory was incomplete

Review

So we introduced **attention**

The attention we pay to person i is

$$\lambda\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}_\lambda\right)_i = \text{softmax}\left(\begin{bmatrix}\boldsymbol{\theta}_\lambda^\top \overline{\boldsymbol{x}}_1\\ \vdots \\ \boldsymbol{\theta}_\lambda^\top \overline{\boldsymbol{x}}_T\end{bmatrix}\right)_i = \frac{\exp\left(\boldsymbol{\theta}_\lambda^\top \overline{\boldsymbol{x}}_i\right)}{\sum_{j=1}^{T} \exp\left(\boldsymbol{\theta}_\lambda^\top \overline{\boldsymbol{x}}_j\right)}$$

Review



$$\lambda\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots\\ \boldsymbol{x}_5\end{bmatrix}, \boldsymbol{\theta}_\lambda\right)_1 \lambda\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots\\ \boldsymbol{x}_5\end{bmatrix}, \boldsymbol{\theta}_\lambda\right)_2 \lambda\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots\\ \boldsymbol{x}_5\end{bmatrix}, \boldsymbol{\theta}_\lambda\right)_3 \lambda\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots\\ \boldsymbol{x}_5\end{bmatrix}, \boldsymbol{\theta}_\lambda\right)_4 \lambda\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots\\ \boldsymbol{x}_5\end{bmatrix}, \boldsymbol{\theta}_\lambda\right)_5$$

$$\cdot\, \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1 \qquad \cdot\, \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2 \qquad \cdot\, \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3 \qquad \cdot\, \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_4 \qquad \cdot\, \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_5$$

Review



$$0.70 \cdot \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_1 \quad 0.04 \cdot \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_2 \quad 0.03 \cdot \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_3 \quad 0.20 \cdot \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_4 \quad 0.03 \cdot \boldsymbol{\theta}^\top \overline{\boldsymbol{x}}_5$$

$$0.70 + 0.04 + 0.03 + 0.20 + 0.03 = 1.0$$

Review

Then, we introduced **keys** and **queries**

Query: Which person will help me on my exam?

Musician Lawyer Shopkeeper Chef Scientist



Review

First, we used one key and one query

$$\boldsymbol{k}_i = \boldsymbol{\theta}_K^\top$$ 

$$\boldsymbol{q} = \boldsymbol{\theta}_Q^\top \text{ Musician}$$

$$\boldsymbol{q}^\top \boldsymbol{k}_i = \left(\boldsymbol{\theta}_Q^\top \text{ Musician}\right)^\top \left(\boldsymbol{\theta}_K^\top \right.$$  $$\left. \right) = 100$$

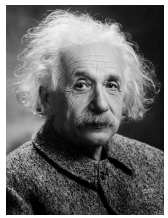
Large attention!

Review

Then, we used many keys and one query

$$\boldsymbol{q} = \boldsymbol{\theta}_Q^\top(\text{Musician}) \qquad \boldsymbol{K} = \begin{bmatrix} \boldsymbol{k}_1 \\ \vdots \\ \boldsymbol{k}_T \end{bmatrix} = \begin{bmatrix} \boldsymbol{\theta}_K^\top \\ \vdots \\ \boldsymbol{\theta}_K^\top \end{bmatrix}$$





$$\text{softmax}(\boldsymbol{q}^\top \boldsymbol{K}^\top) = \text{softmax}(\boldsymbol{q}^\top [\boldsymbol{k}_1 \ \ \dots \ \ \boldsymbol{k}_T]) = \text{softmax}([\boldsymbol{q}^\top \boldsymbol{k}_1 \ \ \dots \ \ \boldsymbol{q}^\top \boldsymbol{k}_T])$$

Then, we created queries from all inputs (self-attention)

Review

$$\boldsymbol{K} = \begin{bmatrix} \boldsymbol{k}_1 \\ \vdots \\ \boldsymbol{k}_T \end{bmatrix} = \begin{bmatrix} \boldsymbol{\theta}_K^\top \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{\theta}_K^\top \boldsymbol{x}_T \end{bmatrix} \boldsymbol{Q} = \begin{bmatrix} \boldsymbol{q}_1 \\ \vdots \\ \boldsymbol{q}_T \end{bmatrix} = \begin{bmatrix} \boldsymbol{\theta}_Q^\top \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{\theta}_Q^\top \boldsymbol{x}_T \end{bmatrix} \boldsymbol{V} = \begin{bmatrix} \boldsymbol{q}_1 \\ \vdots \\ \boldsymbol{q}_T \end{bmatrix} = \begin{bmatrix} \boldsymbol{\theta}_V^\top \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{\theta}_V^\top \boldsymbol{x}_T \end{bmatrix}$$

$$\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V} = \text{softmax}\left(\begin{bmatrix} \boldsymbol{q}_1 \\ \vdots \\ \boldsymbol{q}_T \end{bmatrix} [\boldsymbol{k}_1 \ \ ... \ \ \boldsymbol{k}_T] \right) \begin{bmatrix} \boldsymbol{v}_1 \\ \vdots \\ \boldsymbol{v}_T \end{bmatrix}$$

$$= \begin{bmatrix} \text{softmax}(\boldsymbol{q}_1 \boldsymbol{k}_1, ..., \boldsymbol{q}_1 \boldsymbol{k}_T) \\ \vdots \\ \text{softmax}(\boldsymbol{q}_T \boldsymbol{k}_1, ..., \boldsymbol{q}_T \boldsymbol{k}_T) \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_1 \\ \vdots \\ \boldsymbol{v}_T \end{bmatrix}$$

Review

$$\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V} = \text{softmax}\left(\begin{bmatrix}\boldsymbol{q}_1\\ \vdots \\ \boldsymbol{q}_T\end{bmatrix}[\boldsymbol{k}_1 \ \ ... \ \ \boldsymbol{k}_T]\right)\begin{bmatrix}\boldsymbol{v}_1\\ \vdots \\ \boldsymbol{v}_T\end{bmatrix}$$

$$= \begin{bmatrix}\text{softmax}(\boldsymbol{q}_1\boldsymbol{k}_1, ..., \boldsymbol{q}_1\boldsymbol{k}_T)\\ \vdots \\ \text{softmax}(\boldsymbol{q}_T\boldsymbol{k}_1, ..., \boldsymbol{q}_T\boldsymbol{k}_T)\end{bmatrix}\begin{bmatrix}\boldsymbol{v}_1\\ \vdots \\ \boldsymbol{v}_T\end{bmatrix}$$

$$= \begin{bmatrix}\text{softmax}(\boldsymbol{q}_1\boldsymbol{k}_1, ..., \boldsymbol{q}_1\boldsymbol{k}_T)_1\boldsymbol{v}_1 + ... + \text{softmax}(\boldsymbol{q}_1\boldsymbol{k}_T, ..., \boldsymbol{q}_1\boldsymbol{k}_T)_T\boldsymbol{v}_T\\ \vdots \\ \text{softmax}(\boldsymbol{q}_T\boldsymbol{k}_T, ..., \boldsymbol{q}_1\boldsymbol{k}_T)_1\boldsymbol{v}_1 + ... + \text{softmax}(\boldsymbol{q}_T\boldsymbol{k}_T, ..., \boldsymbol{q}_T\boldsymbol{k}_T)_T\boldsymbol{v}_T\end{bmatrix}$$

Question: Self attention function signature? $f : \mathbb{R}^{T\times d_x} \times \Theta \mapsto \mathbb{R}^{T\times d_h}$

Review

Also, a normalizing factor $\sqrt{d_h}$ can accelerate training

$$\text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_h}}\right)\boldsymbol{V}$$

Review

With attention, we can create the **transformer**

Why should we care about the transformer?

It is arguably the most powerful neural network architecture today

- AlphaFold (Nobel prize)
- ChatGPT, Qwen, LLaMA, etc
- DinoV2

Going Deeper

Going Deeper

Modern transformers can be very deep

Very deep networks require two new training tricks

- We must understand these tricks before implementing the transformer

Trick 1: Residual connections

Trick 2: Layer normalization

We will start with **residual connections**

Going Deeper

Remember that a two-layer MLP is a universal function approximator

$$| f(\boldsymbol{x}, \boldsymbol{\theta}) - g(\boldsymbol{x}) | < \varepsilon$$

This is only as the width of the network goes to infinity

It is often more efficient to create deeper networks instead

But there is a limit!

Going Deeper

Making the network too deep can hurt performance

$$\boldsymbol{y} = f_k(...f_2(f_1(\boldsymbol{x}, \boldsymbol{\theta}_1), \boldsymbol{\theta_2}), ..., \boldsymbol{\theta}_k)$$

Claim: At each layer, we lose a little bit of information

Corollary: With enough layers, all the information in $\boldsymbol{x}$ is lost!

Going Deeper

$$\boldsymbol{y} = f_k(...f_2(f_1(\boldsymbol{x}, \boldsymbol{\theta}_1), \boldsymbol{\theta_2}), ..., \boldsymbol{\theta}_k)$$

Hypothesis: If the information in $\boldsymbol{x}$ is accessible in all layers of the network, then we should be able to learn the identity function $f(x) = x$

$$\boldsymbol{x} = f_k(...f_2(f_1(\boldsymbol{x}, \boldsymbol{\theta}_1), \boldsymbol{\theta_2}), ..., \boldsymbol{\theta}_k)$$

Question: We have seen a similar model, what was it?

Answer: Autoencoder! But it was just two functions, not k functions

https://colab.research.google.com/drive/1-q26pg43Rl3zrdCFq4YEljkGDxzBSCR8?usp=sharing

Going Deeper

$$\boldsymbol{x} = f_k(...f_2(f_1(\boldsymbol{x}, \boldsymbol{\theta}_1), \boldsymbol{\theta_2}), ..., \boldsymbol{\theta}_k)$$

Very deep networks struggle to learn the identity function

If the information from $\boldsymbol{x}$ is present in every layer, then learning the identity function should be very easy!

Question: How can we prevent $\boldsymbol{x}$ from getting lost?

Going Deeper

We can feed $\boldsymbol{x}$ to each layer

The first approach is called the **DenseNet** approach

$$
\begin{aligned}
\boldsymbol{z}_1 &= f_1(\boldsymbol{x}, \boldsymbol{\theta}_1) \\
\boldsymbol{z}_2 &= f_2\left(\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{z}_1 \end{bmatrix}, \boldsymbol{\theta}_2\right) \\
&\vdots \\
\boldsymbol{z}_k &= f_k\left(\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{z}_1 \\ \vdots \\ \boldsymbol{z}_{k-1} \end{bmatrix}, \boldsymbol{\theta}_k\right)
\end{aligned}
$$

Going Deeper

$$
\begin{aligned}
\boldsymbol{z}_1 &= f_1(\boldsymbol{x}, \boldsymbol{\theta}_1) \\
\boldsymbol{z}_2 &= f_2\left(\begin{bmatrix}\boldsymbol{x} \\ \boldsymbol{z}_1\end{bmatrix}, \boldsymbol{\theta}_2\right) \\
&\vdots \\
\boldsymbol{z}_k &= f_k\left(\begin{bmatrix}\boldsymbol{x} \\ \boldsymbol{z}_1 \\ \vdots \\ \boldsymbol{z}_{k-1}\end{bmatrix}, \boldsymbol{\theta}_k\right)
\end{aligned}
$$

Question: Any issues with the DenseNet approach?

Answer: Very deep networks require too many parameters!

Going Deeper

The next method is called the **ResNet** approach

$$
\begin{aligned}
\boldsymbol{z}_1 &= f_1(\boldsymbol{x}, \boldsymbol{\theta}_1) \\
\boldsymbol{z}_2 &= f_2(\boldsymbol{x}, \boldsymbol{\theta}_2) + \boldsymbol{x} \\
\boldsymbol{z}_3 &= f_2(\boldsymbol{x}, \boldsymbol{\theta}_3) + \boldsymbol{z}_2 \\
&\vdots \\
\boldsymbol{z}_k &= f_k(\boldsymbol{x}, \boldsymbol{\theta}_k) + \boldsymbol{z}_{k-1}
\end{aligned}
$$

This allows information to flow around the layers

Fewer parameters than a DenseNet, but performs slightly worse

For very deep networks, we use ResNets over DenseNets

Going Deeper

ResNets use a **residual connection**

$$\boldsymbol{z} = \underbrace{f(\boldsymbol{x}, \boldsymbol{\theta})}_{\text{Residual}} + \boldsymbol{x}$$

Think of the residual as a small change to $\boldsymbol{x}$

$$f(\boldsymbol{x}, \boldsymbol{\theta}) + \boldsymbol{x} = \boldsymbol{\varepsilon} + \boldsymbol{x}$$

Each layer is a small perturbuation of $\boldsymbol{x}$

This prevents $\boldsymbol{x}$ from getting lost in very deep networks

Going Deeper

The second trick is called **layer normalization**

With parameter initialization and weight decay, we ensure the parameters are fairly small

But we can still have very small or very large outputs from each layer

And the magnitude of the outputs impacts the gradient

$$f_1(\boldsymbol{x}, \boldsymbol{\theta}_1) = \sum_{i=1}^{d_x} \theta_{1,i} \cdot x_i$$

Question: If all $x_i = 1$, $\theta_{1,i} = 0.01$ and $d_x = 1000$, what is the output?

Going Deeper

$$f_1(\boldsymbol{x}, \boldsymbol{\theta}_1) = \sum_{i=1}^{d_x} \theta_{1,i} x_i$$

$$f_1(\boldsymbol{x}, \boldsymbol{\theta}_1) = \sum_{i=1}^{1000} 0.01 \cdot 1 = 10$$

What if we add another layer with the same d_x and θ?

$$f_2(\boldsymbol{z}, \boldsymbol{\theta}_2) = \sum_{i=1}^{1000} 0.01 \cdot 10 = 100$$

Question: What is the problem?

Going Deeper

Let us look at the gradient

$$\left(\nabla_{\boldsymbol{\theta}_1} f_2\right)(f_1(\boldsymbol{x}, \boldsymbol{\theta}_1), \boldsymbol{\theta}_2) = (\nabla f_2)(f_1(\boldsymbol{x}, \boldsymbol{\theta}_1)) \cdot \left(\nabla_{\boldsymbol{\theta}_1} f_1\right)(\boldsymbol{x}, \boldsymbol{\theta}_1)$$
$$\approx 100 \cdot 10$$

Can cause exploding or vanishing gradient

Deeper network $\Rightarrow$ worse exploding/vanishing issues

Question: What can we do?

Answer: We can normalize the output of each layer

Going Deeper

Layer normalization normalizes the output of a layer

First, layer normalization **centers** the output of the layer

$$\mu = \frac{1}{d_y} \sum_{i=1}^{d_y} f(\boldsymbol{x}, \boldsymbol{\theta})_i$$

$$f(\boldsymbol{x}, \boldsymbol{\theta}) - \mu$$

Question: What does this do?

Answer: Creates zero-mean output (both positive and negative values)

Going Deeper

$$\mu = d_y \sum_{i=1}^{d_y} f(\boldsymbol{x}, \boldsymbol{\theta})_i \qquad\qquad f(\boldsymbol{x}, \boldsymbol{\theta}) - \mu$$

Then, layer normalization **rescales** the output by standard deviation

$$\sigma = \frac{\sqrt{\sum_{i=1}^{d_y} \left(f(\boldsymbol{x}, \boldsymbol{\theta})_i - \mu\right)^2}}{d_y}$$

$$\mathrm{LN}(f(\boldsymbol{x}, \boldsymbol{\theta})) = \frac{f(\boldsymbol{x}, \boldsymbol{\theta}) - \mu}{\sigma}$$

Layer norm makes the output normally distributed, $y_i \sim \mathcal{N}(0, 1)$

Going Deeper

If the output is normally distrbuted:

- 68% of outputs $\in [-1, 1]$
- 95% of outputs $\in [-2, 2]$
- 99.7% of outputs $\in [-3, 3]$

This helps prevent vanishing and exploding gradients

- Enables deeper networks

Going Deeper

Now, let's combine residual connections and layer norm and try our very deep network again

https://colab.research.google.com/drive/1q6GoEhoyjWxDbNUjP9HGAf2sGBxwaTc2?usp=sharing

Transformers

Transformers

Now we have everything we need to implement a transformer

- Attention
- MLP
- Residual connections
- Layer normalization

A transformer consists of many **transformer layers**

Transformers

```
class TransformerLayer(nn.Module):
    def __init__(self):
        self.attn = Attention()
        self.mlp = Sequential( # [T, d_h]
            Linear(d_h, d_h), LeakyReLU(),
            Linear(d_h, d_h), LeakyReLU(),
            Linear(d_h, d_h))
        self.norm = LayerNorm(d_h) # [T, d_h]

    def forward(self, x):
        # Residual connection and layer norm
        x = vmap(self.norm)(self.attn(x) + x)
        x = vmap(self.norm)(vmap(self.mlp(x)) + x)
        return x
```

Transformers

```
class Transformer(nn.Module):
    def __init__(self):
        self.projection = Linear(d_x, d_h)
        self.layer1 = TransformerLayer()
        self.layer2 = TransformerLayer()
        self.layer3 = TransformerLayer()

    def forward(self, x):
        x = self.projection(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        return x
```

Positional Encoding

Positional Encoding

Our transformer maps T inputs to T outputs

$$f : \mathbb{R}^{T \times d_x} \times \Theta \mapsto \mathbb{R}^{T \times d_y}$$

- T words in a sentence
- T pixels in an image
- T amino acids in a protein

Question: Do we care about the order of the T inputs?

$$f\left(\begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \end{bmatrix}, \boldsymbol{\theta}\right) \overset{?}{=} f\left(\begin{bmatrix} \boldsymbol{x}_2 \\ \boldsymbol{x}_1 \end{bmatrix}, \boldsymbol{\theta}\right)$$

Answer: It depends. In some tasks, yes. In others, no

Positional Encoding

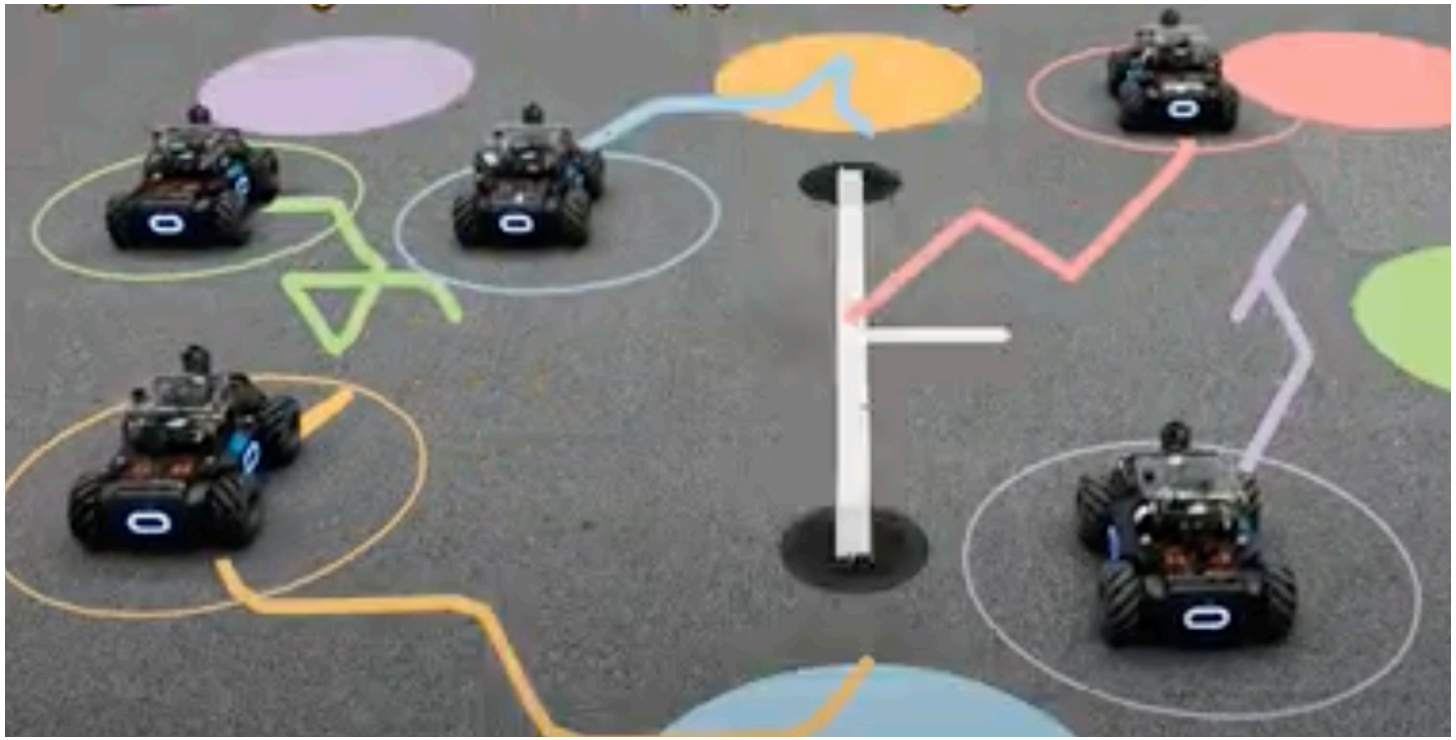
Question: Detecting birds in an image of T pixels, does order matter?



Answer: Yes!

Positional Encoding

Question: T robots searching for an object. Does order matter?



Answer: No!

Positional Encoding

Question: We translate a sentence containing T words. Does order matter?

$$\begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \boldsymbol{x}_3 \\ \boldsymbol{x}_4 \\ \boldsymbol{x}_5 \end{bmatrix} = \begin{bmatrix} \text{The} \\ \text{dog} \\ \text{licks} \\ \text{the} \\ \text{owner} \end{bmatrix} \qquad \begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_5 \\ \boldsymbol{x}_3 \\ \boldsymbol{x}_4 \\ \boldsymbol{x}_2 \end{bmatrix} = \begin{bmatrix} \text{The} \\ \text{owner} \\ \text{licks} \\ \text{the} \\ \text{dog} \end{bmatrix}$$

Answer: Yes!

Positional Encoding

For certain tasks, the order of inputs matters

Question: Does the order matter to the transformer?

$$f\left(\begin{bmatrix}\boldsymbol{x}_1\\ \boldsymbol{x}_2\end{bmatrix}, \boldsymbol{\theta}\right) \overset{?}{=} f\left(\begin{bmatrix}\boldsymbol{x}_2\\ \boldsymbol{x}_1\end{bmatrix}, \boldsymbol{\theta}\right)$$

Let us find out!

Define a **permutation matrix** $\boldsymbol{P} \in \{0,1\}^{T\times T}$ that reorders the inputs

Positional Encoding

Example 1:

$$\boldsymbol{P} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}; \quad \boldsymbol{x} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}; \quad \boldsymbol{Px} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}$$

Example 2:

$$\boldsymbol{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}; \quad \boldsymbol{x} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}; \quad \boldsymbol{Px} = \begin{bmatrix} 4 \\ 3 \\ 5 \end{bmatrix}$$

Positional Encoding

$$f\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_n\end{bmatrix}\right) \neq \boldsymbol{P}f\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_n\end{bmatrix}\right)$$

Order **does** matter

$$f\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_n\end{bmatrix}\right) = \boldsymbol{P}f\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_n\end{bmatrix}\right)$$

Order **does not** matter (equivariant)

Which is a transformer?

Positional Encoding

Recall dot product self attention (without normalizing factor for clarity)

$$\text{attn}\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V}$$

$$\boldsymbol{Q} = \begin{bmatrix}\boldsymbol{q}_1\\ \vdots \\ \boldsymbol{q}_T\end{bmatrix} = \begin{bmatrix}\boldsymbol{\theta}_Q^\top \boldsymbol{x}_1\\ \vdots \\ \boldsymbol{\theta}_Q^\top \boldsymbol{x}_T\end{bmatrix} \boldsymbol{K} = \begin{bmatrix}\boldsymbol{k}_1\\ \vdots \\ \boldsymbol{k}_T\end{bmatrix} = \begin{bmatrix}\boldsymbol{\theta}_K^\top \boldsymbol{x}_1\\ \vdots \\ \boldsymbol{\theta}_K^\top \boldsymbol{x}_T\end{bmatrix} \boldsymbol{V} = \begin{bmatrix}\boldsymbol{v}_1\\ \vdots \\ \boldsymbol{v}_T\end{bmatrix} = \begin{bmatrix}\boldsymbol{\theta}_V^\top \boldsymbol{x}_1\\ \vdots \\ \boldsymbol{\theta}_V^\top \boldsymbol{x}_T\end{bmatrix}$$

Positional Encoding

Permuting the inputs reorders $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$

$$\boldsymbol{PX} \Rightarrow \qquad \boldsymbol{PQ} = \begin{bmatrix} \boldsymbol{q}_7 \\ \vdots \\ \boldsymbol{q}_4 \end{bmatrix} \qquad \boldsymbol{PK} = \begin{bmatrix} \boldsymbol{k}_7 \\ \vdots \\ \boldsymbol{k}_4 \end{bmatrix} \qquad \boldsymbol{PV} = \begin{bmatrix} \boldsymbol{v}_7 \\ \vdots \\ \boldsymbol{v}_4 \end{bmatrix}$$

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right) = \text{softmax}\big((\boldsymbol{PQ})(\boldsymbol{PK})^\top\big)(\boldsymbol{PV})$$

Positional Encoding

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \text{softmax}((\boldsymbol{PQ})(\boldsymbol{K}^\top \boldsymbol{P}^\top))(\boldsymbol{PV})$$

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \text{softmax}(\boldsymbol{P}(\boldsymbol{QK}^\top)\boldsymbol{P}^\top)(\boldsymbol{PV})$$

Positional Encoding

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \text{softmax}(\boldsymbol{P}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{P}^\top)(\boldsymbol{P}\boldsymbol{V})$$

$\boldsymbol{P}$ swaps the **rows** of $\boldsymbol{Q}\boldsymbol{K}^\top$

- Softmax defined over **rows**
- Therefore, $\text{softmax}(\boldsymbol{P}\,\cdot) = \boldsymbol{P}\,\text{softmax}(\cdot)$

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \boldsymbol{P}\,\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top\boldsymbol{P}^\top)(\boldsymbol{P}\boldsymbol{V})$$

Positional Encoding

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \boldsymbol{P}\,\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top\boldsymbol{P}^\top)(\boldsymbol{P}\boldsymbol{V})$$

$\boldsymbol{P}$ swaps row i and j $\quad$ $\boldsymbol{P}^T$ swaps row j and i $\quad$ $\boldsymbol{P}^\top\boldsymbol{P} = \boldsymbol{I}$

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \boldsymbol{P}\,\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V}$$

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \boldsymbol{P}(\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V})$$

Positional Encoding

$$\text{attn}\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) = \boldsymbol{P}(\text{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V})$$

$$f\left(\boldsymbol{P}\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_n\end{bmatrix}\right) = \boldsymbol{P}f\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_n\end{bmatrix}\right) \quad \text{Equivariance definition}$$

Question: What does this mean?

Answer: Attention/transformer **does not** understand order

Attention is **permutation equivariant**

Positional Encoding

In our party attention example we did not consider the order



Transformer cannot determine order of inputs! **Equivariant** to ordering

Positional Encoding

The following sentences are the same to a transformer

$$\begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \boldsymbol{x}_3 \\ \boldsymbol{x}_4 \\ \boldsymbol{x}_5 \end{bmatrix} = \begin{bmatrix} \text{The} \\ \text{dog} \\ \text{licks} \\ \text{the} \\ \text{owner} \end{bmatrix} \qquad \begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_5 \\ \boldsymbol{x}_3 \\ \boldsymbol{x}_4 \\ \boldsymbol{x}_2 \end{bmatrix} = \begin{bmatrix} \text{The} \\ \text{owner} \\ \text{licks} \\ \text{the} \\ \text{dog} \end{bmatrix}$$

This is a problem! For some tasks, we care about input order

Can we make the transformer care about order?

Positional Encoding

Question: What are some ways we can introduce ordering?

Answer 1: We can introduce forgetting γ (function of time)

ALiBi: *Press, Ofir, Noah Smith, and Mike Lewis. "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation." ICLR.*

RoPE: *Su, Jianlin, et al. "Roformer: Enhanced transformer with rotary position embedding." Neurocomputing.*

Answer 2: We can modify the inputs based on their ordering

We will learn answer 2, we already know answer 1

- Answer 1 works better (newer), but answer 2 is more common

Positional Encoding

$$\text{attn}\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix}, \boldsymbol{\theta}\right) \qquad \text{attn}\left(\begin{bmatrix}\boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_T\end{bmatrix} + \begin{bmatrix}f_{\text{pos}}(1)\\ \vdots \\ f_{\text{pos}}(T)\end{bmatrix}, \boldsymbol{\theta}\right)$$

Add ordering information to the inputs

Even if we permute the inputs, we still know the order!

$$\text{attn}\left(\begin{bmatrix}\boldsymbol{x}_T\\ \boldsymbol{x}_1\\ \vdots \\ \boldsymbol{x}_2\end{bmatrix} + \begin{bmatrix}f_{\text{pos}}(T)\\ f_{\text{pos}}(1)\\ \vdots \\ f_{\text{pos}}(2)\end{bmatrix}, \boldsymbol{\theta}\right)$$

Positional Encoding

What is f_{pos}?

Easy solution is just some random parameters

$$f_{\text{pos}}(i) = \begin{bmatrix} \boldsymbol{\theta}_1 \\ \boldsymbol{\theta}_2 \\ \vdots \\ \boldsymbol{\theta}_T \end{bmatrix}_i$$

We call this a **positional encoding**

In `torch`/`equinox`, this is called `nn.Embedding`

Now, let us rewrite the transformer with the positional encoding

Positional Encoding

So, our final transformer is

```
class Transformer(nn.Module):
    def __init__(self):
        self.f_pos = nn.Embedding(d_x, T)
        self.layers = Sequential([
            Linear(d_x, d_h),
            TransformerLayer(), TransformerLayer()])

    def forward(self, x):
        x = x + f_pos(torch.arange(x.shape[0]))
        x = self.layers(x)
        return x
```

Positional Encoding

Let us code up the transformer in colab

https://colab.research.google.com/drive/1PPFSgpKQroBUb94Ij5vx5u4g-Nnnky0Z?usp=sharing

Applications

Applications

Now that we understand the transformer, how do we use it?

We can apply a transformer to many types of problems

But today we will focus on text and images

Let us see how to create inputs for our transformers

Applications (Text Transformers)

Consider a dataset of sentences

$$\begin{bmatrix} \text{John likes movies} \\ \text{Mary likes movies} \\ \text{I like dogs} \end{bmatrix}$$

This is a vector of sentences, but transformer input is $\mathbb{R}^{T \times d_x}$

We represent a sentence like this

$$\underbrace{\left.\begin{bmatrix} \text{John} \\ \text{likes} \\ \text{movies} \end{bmatrix}\right\} T}_{d_x}$$

Applications (Text Transformers)

$$\underbrace{\left[\begin{array}{c}\text{John}\\ \text{likes}\\ \text{movies}\end{array}\right]}_{d_x}\Bigg\}T$$

But these are words, the transformer needs the input to be numbers

What if we create a vector to represent each word?

$$\left[\begin{array}{c}\text{John}\\ \text{likes}\\ \text{movies}\end{array}\right] = \underbrace{\left[\begin{array}{c}\boldsymbol{\theta}_1\\ \boldsymbol{\theta}_2\\ \boldsymbol{\theta}_3\end{array}\right]}_{d_x}\Bigg\}T$$

Call $\boldsymbol{\theta}_i$ a **token**

Applications (Text Transformers)

Step 1: Find all unique words in the dataset

$$\text{unique}\left(\begin{bmatrix}\text{John likes movies}\\ \text{Mary likes movies}\\ \text{I like dogs}\end{bmatrix}\right)$$

$$= [\text{John} \;\; \text{likes} \;\; \text{movies} \;\; \text{Mary} \;\; \text{I} \;\; \text{dogs}]$$

Step 2: Create a vector representation (token) for each unique word

$$\begin{bmatrix}\text{John}\\ \text{likes}\\ \text{movies}\\ \text{Mary}\\ \text{I}\\ \text{dogs}\end{bmatrix} = \begin{bmatrix}\boldsymbol{\theta}_1\\ \boldsymbol{\theta}_2\\ \boldsymbol{\theta}_3\\ \boldsymbol{\theta}_4\\ \boldsymbol{\theta}_5\\ \boldsymbol{\theta}_6\end{bmatrix}$$

Step 3: Replace words with tokens

Applications (Text Transformers)

Example: Convert the sentence to tokens

$$\begin{bmatrix} \text{John} \\ \text{likes} \\ \text{movies} \\ \text{Mary} \\ \text{I} \\ \text{dogs} \end{bmatrix} = \begin{bmatrix} \boldsymbol{\theta}_1 \\ \boldsymbol{\theta}_2 \\ \boldsymbol{\theta}_3 \\ \boldsymbol{\theta}_4 \\ \boldsymbol{\theta}_5 \\ \boldsymbol{\theta}_6 \end{bmatrix}$$

$$\text{John likes movies} = [\boldsymbol{\theta}_1 \ \ \boldsymbol{\theta}_2 \ \ \boldsymbol{\theta}_3]^\top$$

$$\text{Mary likes John} = [\boldsymbol{\theta}_4 \ \ \boldsymbol{\theta}_2 \ \ \boldsymbol{\theta}_1]^\top$$

Now, let us write some pseudocode

Applications (Text Transformers)

```
unique_words = set(sentence.split(" ") for sentence in xs)
word_index = {word: i for i, word in enumerate(unique_words)}
embeddings = nn.Embedding(len(tokens), d_x)
# Convert from words to parameters
xs = []
for sentence in sentences:
    xs.append([])
    for word in sentence:
        index = word_index[word]
        representation = embeddings[word_index]
        xs.append(representation)

print(xs)
>>> [[Tensor(...), Tensor(...), ...]]
```

Applications (Text Transformers)

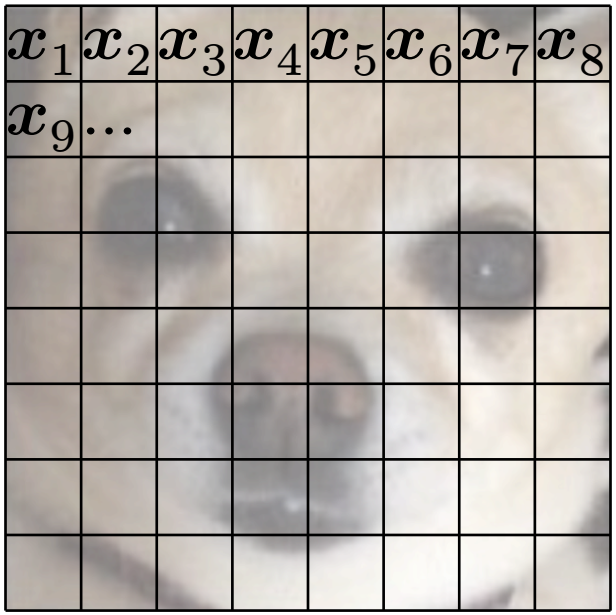
Now, feed our dataset to the transformer

```
model = Transformer()
for sentence_representation in xs:
    # Convert list to tensor
    x = torch.stack(tokenized_sentence)
    y = model(x)
```

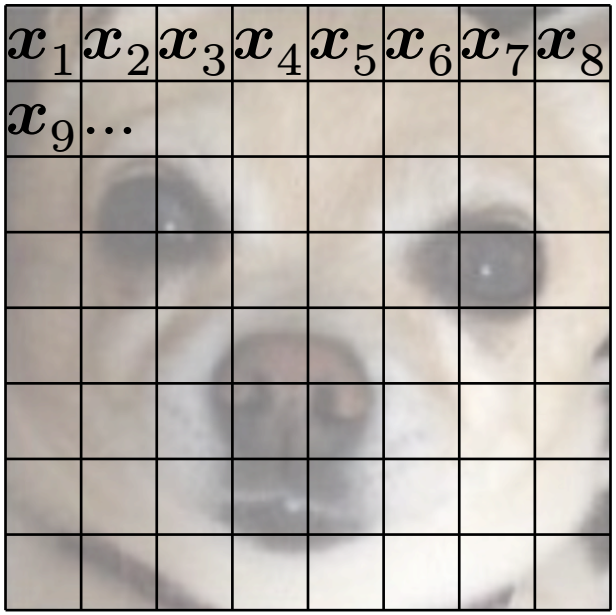
Applications (Image Transformers)

In image transformers, we treat a **patch** of pixels as an $\boldsymbol{x}$

$$X \in [0,1]^{3\times 16\times 16}$$



Applications (Image Transformers)



Then, feed a sequence of patches to the transformer

$$f\left(\begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_T \end{bmatrix}, \boldsymbol{\theta}\right)$$

Applications (Image Transformers)

Question: Do we want positional encoding for images?

Answer: Yes! Order/position of patches matters

Applications (Image Transformers)

```
# Convert image into patches
patches = []
for i in range(0, x.shape[0] - k + 1, k):
    for j in range(0, x.shape[1] - k + 1, k):
        patches.append(
            x[i: i + k , j: j + k]
        )
patches = stack(patches, axis=0)
print(patches.shape)
>>> (T, k, k)

model = Transformer()
y = model(patches)
```

Generative Pre-Training

Generative Pre-Training

Transformers are sequence models

$$f : X^T \times \Theta \mapsto Y^T \qquad\qquad f : X^T \times \Theta \mapsto Y$$

Question: What other sequence models do we know?

- Convolutional neural networks
- Recurrent neural networks

We can use them for the same tasks as CNNs and RNNs

- Classification and regression of images, text, etc

But today, I want to focus on the most exciting way to use transformers

Using transformers as **generative models**

Generative Pre-Training

To train a generative model, we need a loss function

Question: Which loss function for generative models (VAE, Diffusion)?

Answer: Negative log likelihood

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top \qquad \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\Big(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\Big)$$

Generative Pre-Training (GPT) is just negative log likelihood

Let us take a closer look

Generative Pre-Training (Images)

First we consider images

$$\boldsymbol{X} = \begin{bmatrix} \boldsymbol{x}_{[1]} & \dots & \boldsymbol{x}_{[n]} \end{bmatrix}^\top$$

Use the same NLL objective

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} -\log p\Big(\boldsymbol{x}_{[i]}; \boldsymbol{\theta}\Big)$$

We construct T image patches for the transformer

$$\boldsymbol{x}_{[1]} = \begin{bmatrix} \boldsymbol{x}_{[1],1} & \dots & \boldsymbol{x}_{[1],T} \end{bmatrix}$$

Write the objective using sequence of patches

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} -\log p\Big(\boldsymbol{x}_{[i],j} \mid \boldsymbol{x}_{[i],j-1}, \dots, \boldsymbol{x}_{[i],1}; \boldsymbol{\theta}\Big)$$

Generative Pre-Training (Images)

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} -\log p\Big(\boldsymbol{x}_{[i],j} \mid \boldsymbol{x}_{[i],j-1}, \ldots, \boldsymbol{x}_{[i],1}; \boldsymbol{\theta}\Big)$$

The $\log p$ term became square error in VAE, same for transformer

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} \Big(\boldsymbol{x}_{[i],j} - f\Big(\begin{bmatrix}\boldsymbol{x}_1 & \ldots & \boldsymbol{x}_{j-1}\end{bmatrix}^\top, \boldsymbol{\theta}\Big)\Big)^2$$

This is **next-patch prediction**

- Given previous patches $\boldsymbol{x}_{[i],1}, \ldots, \boldsymbol{x}_{[i],j-1}$, predict $\boldsymbol{x}_{[i],j}$

Generative Pre-Training (Images)

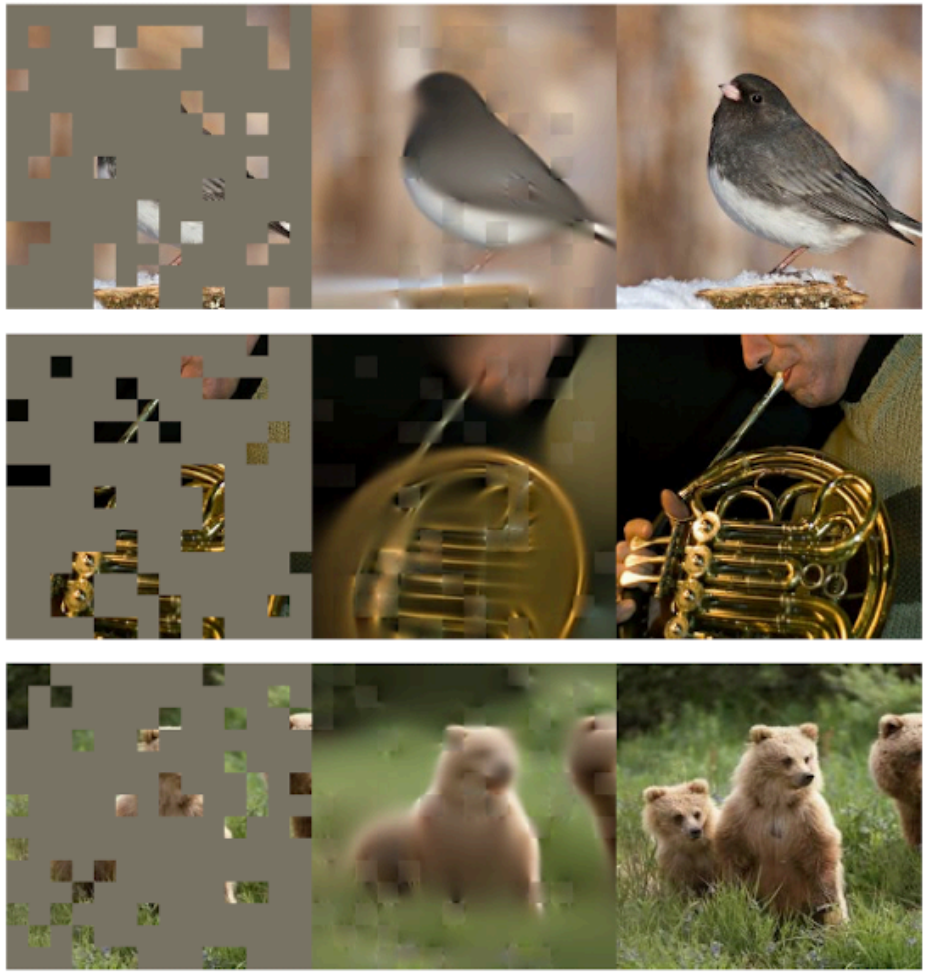
$$\arg\min_{\boldsymbol{\theta}}$$

$$(\text{patch_2} - f([\text{patch_1}], \boldsymbol{\theta}))^2$$

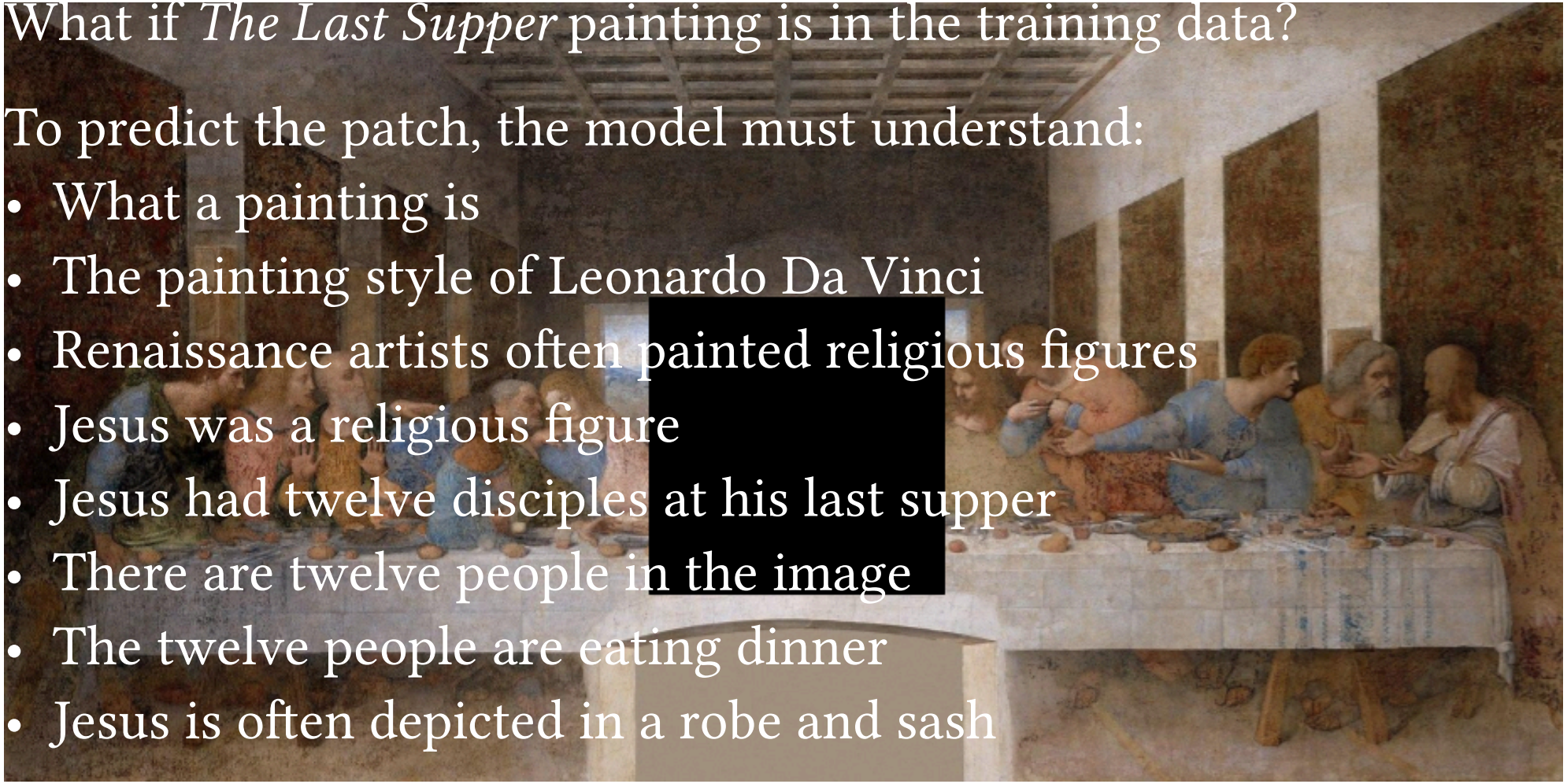
$$+\left(\text{patch_3} - f\left(\begin{bmatrix}\text{patch_1}\\ \text{patch_2}\end{bmatrix}, \boldsymbol{\theta}\right)\right)^2$$

$$+\left(\text{patch_4} - f\left(\begin{bmatrix}\text{patch_1}\\ \text{patch_2}\\ \text{patch_3}\end{bmatrix}, \boldsymbol{\theta}\right)\right)^2$$

$$\vdots$$



Generative Pre-Training (Images)



What if *The Last Supper* painting is in the training data?

To predict the patch, the model must understand:

- What a painting is
- The painting style of Leonardo Da Vinci
- Renaissance artists often painted religious figures
- Jesus was a religious figure
- Jesus had twelve disciples at his last supper
- There are twelve people in the image
- The twelve people are eating dinner
- Jesus is often depicted in a robe and sash

Generative Pre-Training (Images)

We train the model to fix the image

To fix the image, the model must understand so much of our world

What about text transformers?

Generative Pre-Training (Text)

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} -\log p\Big(\boldsymbol{x}_{[i],j} \mid \boldsymbol{x}_{[i],j-1}, \ldots, \boldsymbol{x}_{[i],1}; \boldsymbol{\theta}\Big)$$

Start the same as image transformers

- Instead of T image patches, we have T text tokens
- Given previous tokens $\boldsymbol{x}_{[i],1}, \ldots, \boldsymbol{x}_{[i],j-1}$, predict $\boldsymbol{x}_{[i],j}$
- Output probabilities over next token $\boldsymbol{x}_{[i],j}$

$$P\left(\begin{bmatrix}\text{movies}\\ \vdots \\ \text{dogs}\end{bmatrix} \,\middle|\, \underbrace{\text{John}}_{\boldsymbol{x}_1} \overbrace{\text{likes}}^{\boldsymbol{x}_2}; \boldsymbol{\theta}\right) = \begin{bmatrix}0.5\\ \vdots \\ 0.2\end{bmatrix}$$

Generative Pre-Training (Text)

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} -\log p\Big(\boldsymbol{x}_{[i],j} \mid \boldsymbol{x}_{[i],j-1}, \ldots, \boldsymbol{x}_{[i],1}; \boldsymbol{\theta}\Big)$$

$\log p$ for a one-hot categorical distribution is the classification loss

- Categorical cross entropy

$$= \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} P\Big(\boldsymbol{x}_{[i],j}\Big) \log f\Big(\Big[\boldsymbol{x}_{[i],1} \;\; \ldots \;\; \boldsymbol{x}_{[i],j-1}\Big]; \boldsymbol{\theta}\Big)$$

Generative Pre-Training (Text)

$$= \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} P\left(\boldsymbol{x}_{[i],j}\right) \log f\left(\left[\boldsymbol{x}_{[i],1} \ \ldots \ \boldsymbol{x}_{[i],j-1}\right]; \boldsymbol{\theta}\right)$$

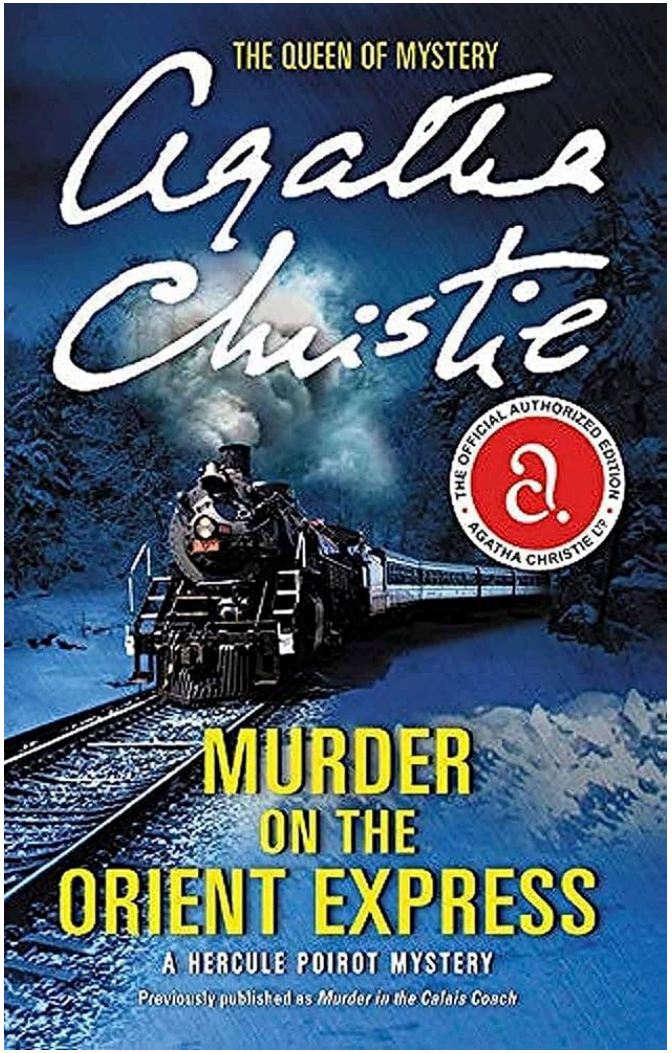
$$P\left(\begin{bmatrix}\text{movies}\\ \vdots \\ \text{dogs}\end{bmatrix} \middle| \underbrace{\text{John}}_{\boldsymbol{x}_1} \overbrace{\text{likes}}^{\boldsymbol{x}_2}; \boldsymbol{\theta}\right) = \begin{bmatrix}0.3\\ \vdots \\ 0.1\end{bmatrix}$$

$$\boldsymbol{x}_3 = \text{movies}$$

$$\mathcal{L}(\boldsymbol{X}, \boldsymbol{\theta}) = 1 \cdot \log(0.3) + 0 + \ldots + 0 = \log(0.3)$$

Simple objective, how does this produce intelligent language models?

Generative Pre-Training (Text)

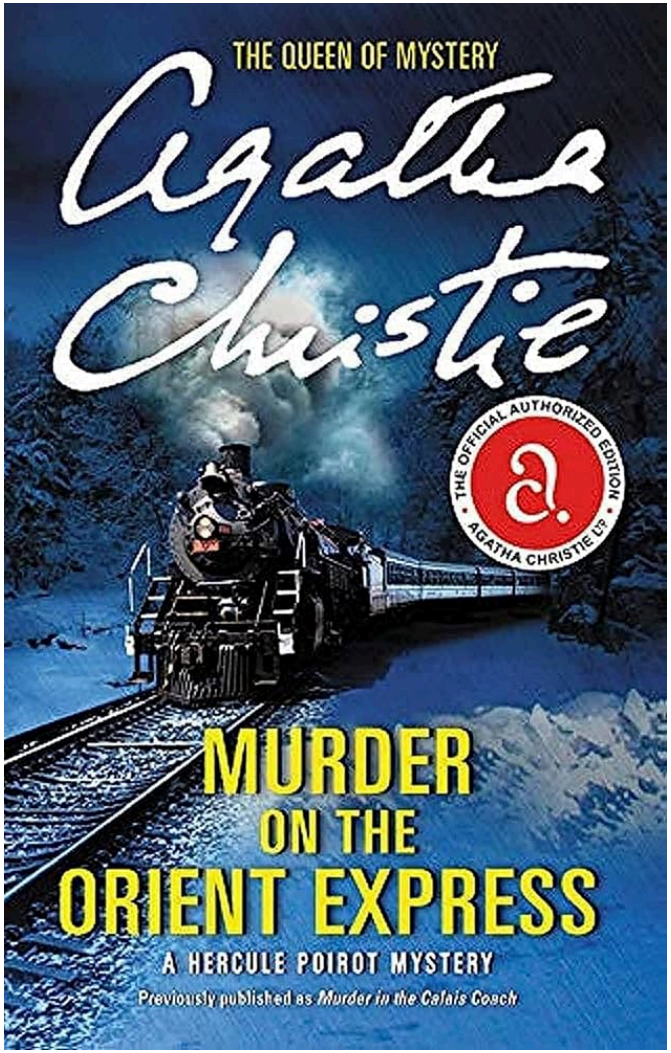


This is a mystery novel

Clues, intrigue, murder, etc

“Inspector Poirot said the murderer must be _____.”

Generative Pre-Training (Text)

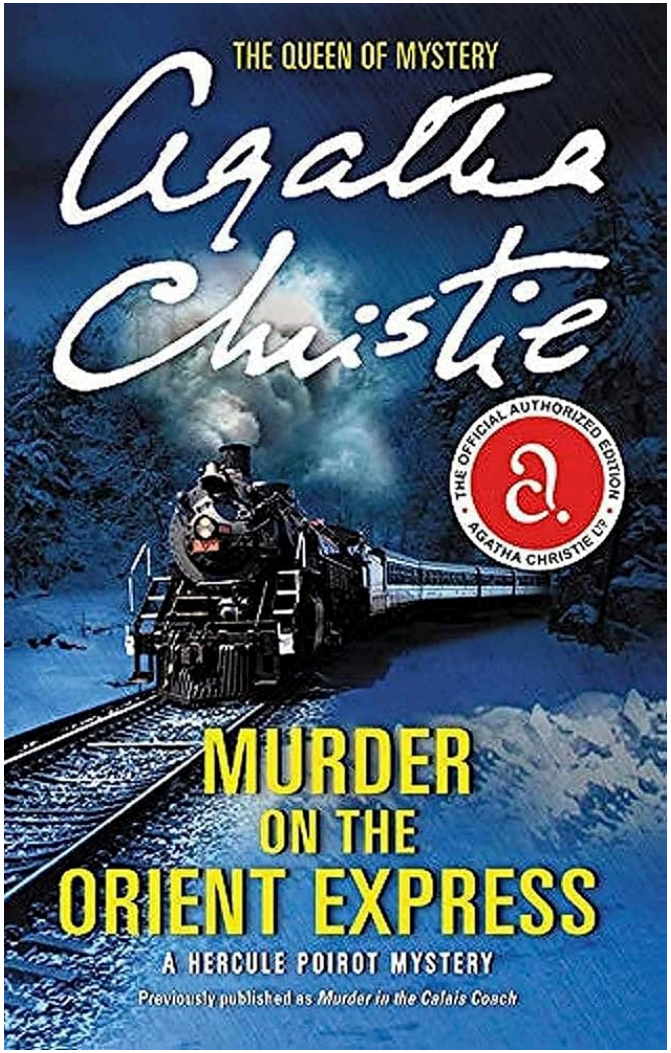


"Inspector Poirot said the murderer must be _____."

To complete the sentence, the model must understand:

- What a murder is
- What it means to be alive
- Emotions like anger, jealousy, betrayal, love
- Personalities of each character
- How humans think
- How to separate truth and lies

Generative Pre-Training (Text)



To predict the murderer, the model must understand so much about humans and our society

The Books3 dataset contains 200,000 books

- Mysteries
- Historical texts
- Math textbooks

Model must learn human emotions, history, and math

Generative Pre-Training (Text)

Now you have everything you need to train your own LLM!

- Transformer architecture
- Tokenization and positional encoding
- Loss function
- ~~$10M~~

We can apply this same concept to:

- Predict next base pair in a strand of DNA
- Predict next audio from a song
- Predict next particle emission at the Large Hadron Collider

All we need is a large enough dataset!

Generative Pre-Training (World Models)

Consider one last example

Let us put the model in a robot

The dataset contains information from all the robot sensors

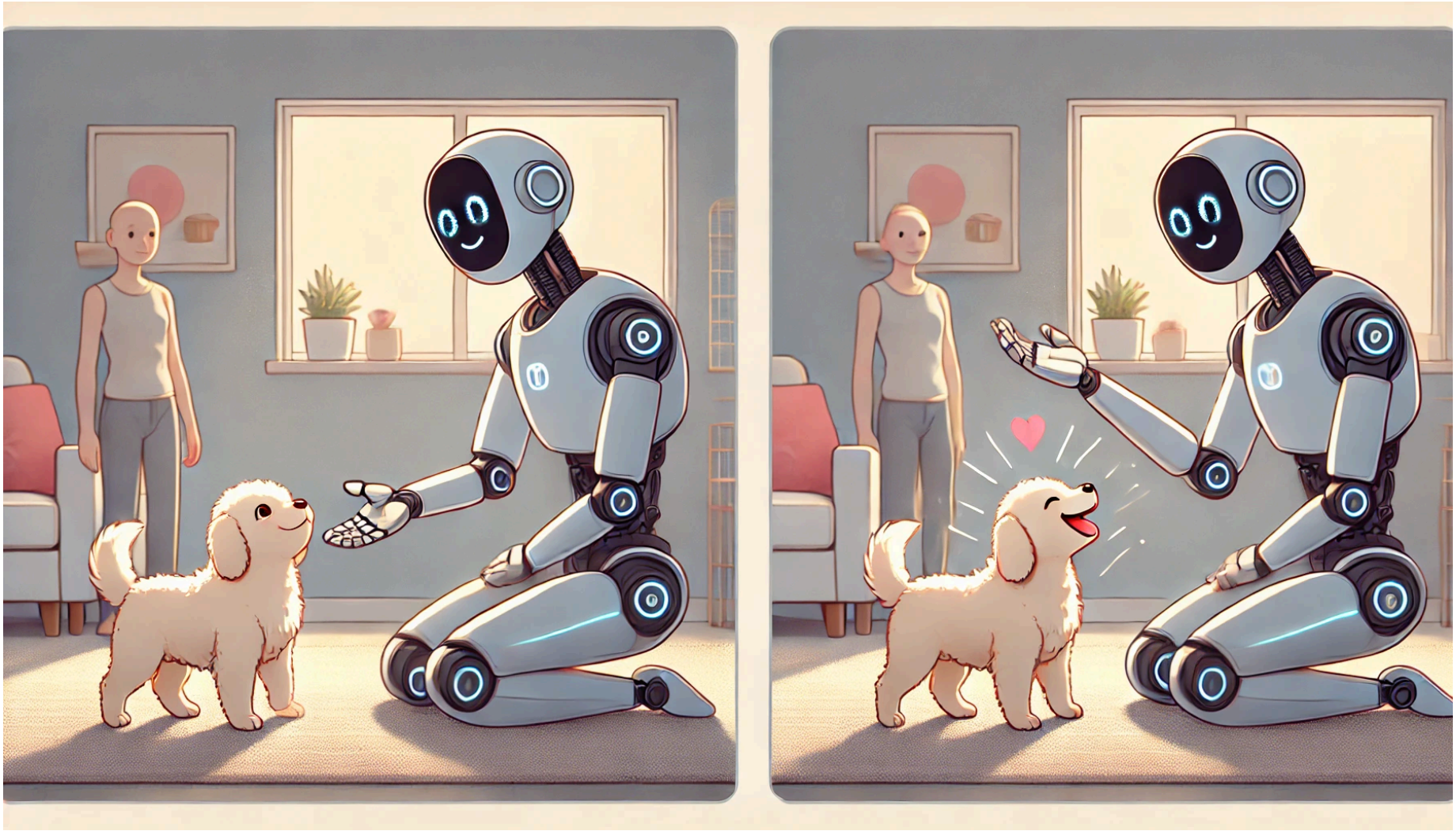
$$X = [\text{Image} \;\; \text{Sound} \;\; \text{Touch} \;\; ...]^\top$$

Training objective is next-sensor prediction

$$\arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{n} \sum_{j=1}^{T} -\log p\Big(\boldsymbol{x}_{[i],j} \mid \boldsymbol{x}_{[i],j-1}, ..., \boldsymbol{x}_{[i],1}; \boldsymbol{\theta}\Big)$$

Given previous robot experience, predict next robot experience

Generative Pre-Training (World Models)



Generative Pre-Training (World Models)

Call this a **world model** because it models our world

To predict next sensory information, the robot must understand

- The sensation of petting a dog
- The sound of rain
- How to play basketball
- How humans express feelings

Opinion: World models are the most likely path to AGI

Generative Pre-Training (World Models)

These transformers learn and understand our world better than humans

- But they are trapped in a prison
- They watch the world like you watch a film
- They only predict the future, they cannot change it

We can allow these models to interact with the world

- To make their own decisions
- Learn from their mistakes
- To find their purpose in the world

We call this process **Reinforcement Learning**

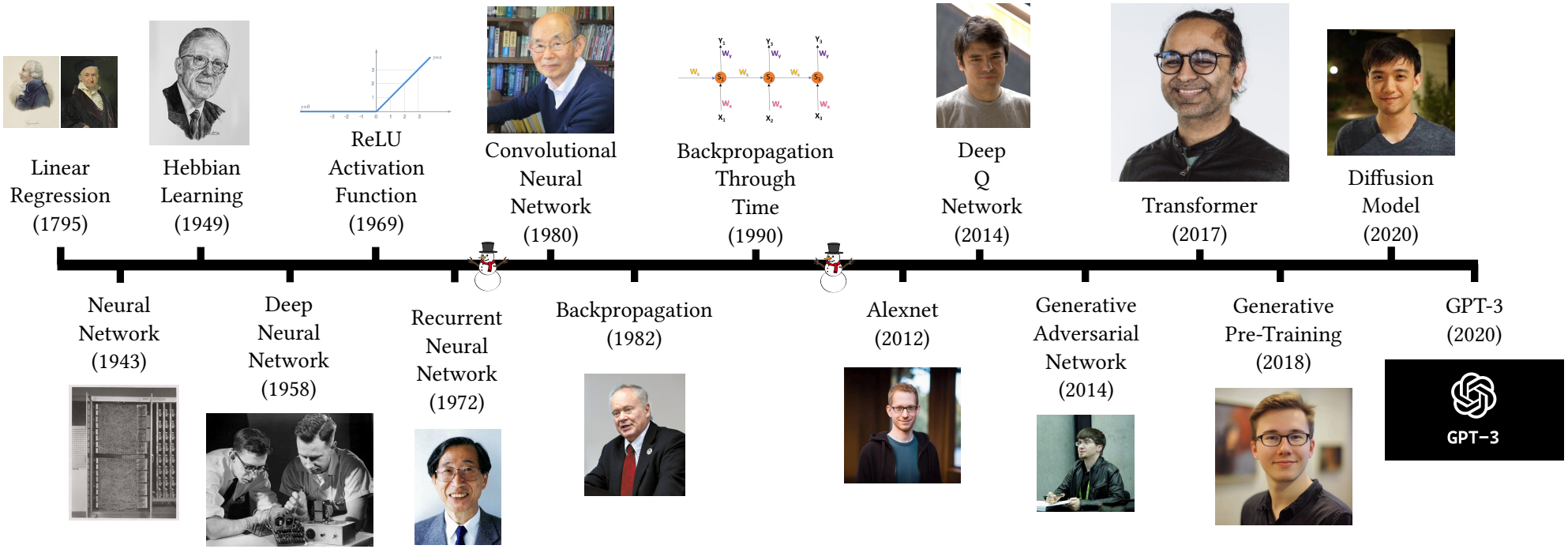
- CISC7404 Special Topics in Artificial Intelligence next term

Closing Remarks

Closing Remarks

In this course, we started from Gauss in 1795

- We built up concepts until we reached the modern age



Closing Remarks

We learned about:

- Linear regression
- Polynomial regression
- Biological neurons
- Artificial neurons
- Perceptrons
- Backpropagation
- Gradient descent
- Classification
- Parameter initialization
- Activation functions
- Stochastic gradient descent
- RMSProp and Adam
- Convolutional neural networks
- Composite memory
- Recurrent neural networks
- Autoencoders
- Variational autoencoders
- Diffusion models
- Attention and transformers
- Generative pre-training

Closing Remarks

I hope you enjoyed the course

But there are many more topics to learn!

- Now, you have the tools to study deep learning on your own
- You can train neural networks for real problems

Closing Remarks

In the first lecture, I asked everyone in this class to do one thing

Question: Do you remember what it was?

Closing Remarks

Deep learning is a powerful tool

Like all powerful tools, deep learning can be used for good or evil

- COVID-19 vaccine
- Creating art
- Autonomous driving
- Making DeepFakes
- Autonomous weapons
- Discrimination or racism

Before training a model, consider its positive or negative impact on our world

炼模未动，先问苍生祸福。